CHAPTER 6 | FILE ORGANISATION & STRUCTURE

Sailesh Singh

File Organization

It refers to the way records are physically arranged on the storage disk and how they are mapped into blocks. Choosing the right method is crucial because it directly impacts the performance of data retrieval, insertion, and deletion.

It’s types are:

- Heap File Organization
- Sequential File Organization
- B+ Tree File Organization

Heap File Organization

This is the simplest form of file organization.

- Mechanism: Records are placed in the file in the order they arrive. No specific ordering is maintained.
- Pros: Fast data insertion; very efficient for small tables.
- Cons: Searching is slow (requires a linear search); deleting records can leave unused "holes" in the disk.

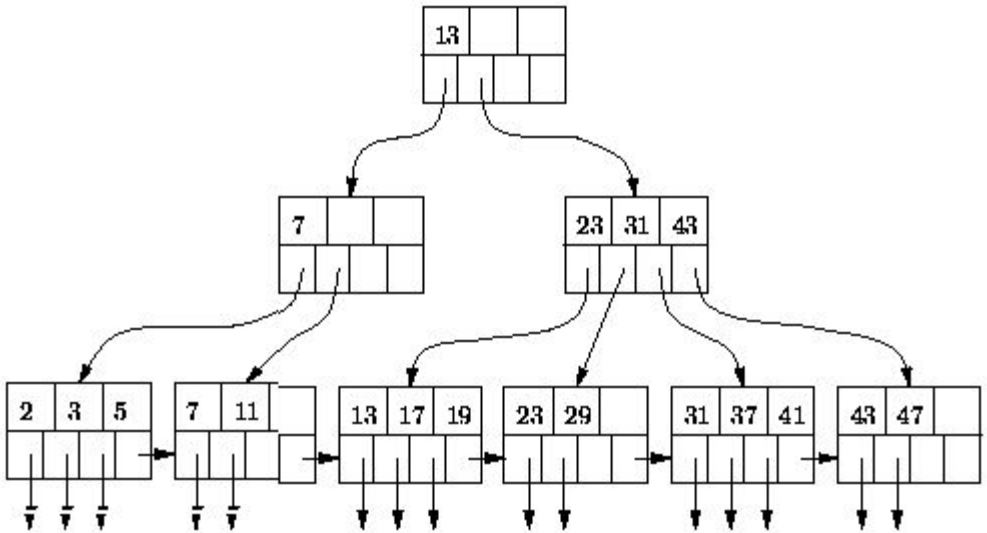
B+ Tree File Organization

This is the most widely used file organization in modern relational databases (like MySQL or PostgreSQL).

- Mechanism: Data is stored in a tree-like hierarchy where the leaf nodes contain the actual data or pointers to it.
- Pros: Highly efficient for both exact matches and range queries; remains balanced even as data grows.
- Cons: More complex to implement and requires extra overhead to keep the tree balanced during updates.

What is a B+ tree?

- Balanced tree data structure
- All leaves at same depth
- Internal nodes:
  Store only keys + pointers
- Leaf nodes:
  Store keys + record pointers (or data)



Searching in B+ Tree Indices

Mode	Target	Returns
Exact Match	Single value	0-1 records
Range Scan	Values between x and y	Multiple records
Prefix Search	Values starting with pattern	Multiple records

Search Complexity:

- I/O Operations: O(log n) node traversals
- Disk Reads: Height of tree (typically 2-4 levels)
- CPU Time: Binary search within each node

Sequential File Organization

In this method, records are stored in a specific order based on a search key (like an ID number).

- Mechanism: Records are linked together or stored physically next to each other in sorted order.
- Pros: Extremely fast for range queries
- Cons: Inserting or deleting records is expensive because the system must maintain the sorted order, often requiring shifting many records.

Records

In DBMS, records are the building blocks of files.

How these records are structured (their "length") dictates how efficiently the database can find, insert, or delete data.

It can be classified into:

- Fixed Length Records
- Variable Length Records

Types of Records | Fixed Length Records

These records can vary in size. This happens when fields have different lengths (like VARCHAR), some fields are optional (NULLs), or records contain repeating items (arrays).

- Pros:
  - Fast Access: The address of the i-th record can be calculated instantly using the formula: Address = Start + (i-1) * RecordSize.
  - Simple Deletion: When a record is deleted, the "hole" is the exact size needed for any new record.
  - Lower Overhead: No need for extra "markers" to tell the system where a record ends.
- Cons:
  - Storage Wastage: If a field (like Name) is set to 50 characters but only uses 10, the remaining 40 bytes are wasted (internal fragmentation).
  - Data Truncation: If data exceeds the fixed limit, it must be cut off.

Types of Records | Variable Length Records

These records can vary in size. This happens when fields have different lengths (like VARCHAR), some fields are optional (NULLs), or records contain repeating items (arrays).

- Pros:
  - Space Efficiency: Only the actual data is stored; no "empty" padding.
  - Flexibility: Supports complex data types and varying field sizes without truncation.
- Cons:
  - Complex Access: You cannot calculate a record's position mathematically. The system must scan or use a pointer directory.
  - Expensive Updates: If a record grows during an update, it may no longer fit in its original spot and must be moved.
  - External Fragmentation: Deleting a 100-byte record leaves a hole that cannot fit a 110-byte record, leading to wasted gaps over time.

Indexing

Indexing is a data structure technique used to quickly locate and access data in a database without scanning every row in a table.

An index consists of a collection of index entries (also called index records). Each entry has two parts:

- Search Key: A copy of the data from the column you want to index (e.g., an Employee ID or Last Name).
- Data Pointer: A reference (usually a physical disk address or a primary key value) that tells the DBMS exactly where the rest of the row is stored on the disk.

Clustered Index

An index where the search key defines the sequential, physical order of the records within the data file.

- Physical Storage: Requires the data file to be physically sorted by the indexing field; therefore, a data file can have at most one clustered index.
- Density: Often sparse, containing one entry for each disk block (using a block anchor) rather than every individual record.
- Efficiency and Performance: Sequential scans are highly efficient because the physical order matches the index order, minimizing disk arm movement.
- Storage and Search Time: Generally requires less storage space and offers shorter search times compared to secondary indexes due to having fewer entries.
- Maintenance Overhead: Can be expensive to maintain during updates; insertions or deletions may cause record relocation (like leaf-node splits) to maintain physical order.

Secondary (Non-Clustered) Index

An index specified on a non-ordering field where the search key specifies a logical order that is different from the physical arrangement of records on the disk.

- Physical Storage: Records are physically scattered relative to the index key; a data file can have multiple non-clustered indexes.
- Density: Typically dense, requiring an entry for every search key value or every record in the data file.
- Efficiency and Performance: Scanning is significantly slower because each record access may require fetching a new, non-contiguous disk block.
- Storage and Search Time: Requires more storage space and longer search times than clustered indexes, though still faster than linear searches.
- Maintenance Overhead: If records are relocated in the clustered index, these indexes must be updated unless they use logical pointers (pointing to the primary key) to provide an additional level of indirection.

RAID (Redundant Array of Independent Disks)

- It is a virtualization technology that combines multiple physical hard drives into a single logical unit.
- Instead of relying on one massive, expensive disk, you use several smaller ones to act as a single, high-performance, and safer storage system.

Foundations of RAID

1. Data Striping

   Splitting data into "chunks" (stripes) and writing them sequentially across multiple physical disks.

2. Data Mirroring

   Maintaining an identical copy (mirror) of data on a separate disk. Every write operation is duplicated.

3. Parity

   A more storage-efficient form of redundancy than mirroring. Parity information (a calculated value based on the data across other disks) is stored.

RAID Controller

This is the hardware/software that manages the RAID array. It's the brain that implements the striping, mirroring, and parity logic.

- Hardware RAID Controller

  A specialized physical device containing dedicated processors, memory, and firmware that manages RAID operations independently of the host system's CPU and operating system.

- Software RAID Controller

  A RAID implementation where all RAID calculations (striping, mirroring, parity) are performed by the host computer's CPU using operating system drivers and software.

RAID Levels

1. RAID Level 0
2. RAID Level 1
3. RAID Level 5
4. RAID Level 6
5. RAID Level 10

RAID Level 0 : Striping

- Data split into blocks ("stripes") across multiple disks
- Simple implementation: Easy to configure and manage
- Zero fault tolerance

    I.e. One disk failure = complete data loss

- Not suitable for critical data

RAID Level 1 : Mirroring

- Exact duplicate of data on two or more disks
- 100% redundancy - Can survive loss of all but one disk
- Capacity: 50% of total disk space (2-disk configuration)
- Excellent fault tolerance: Simple recovery (just copy)
- Simple to understand and manage
- High storage cost: 100% overhead (2× storage for 1× capacity)

RAID Level 5 : Striping with Distributed Parity

- Striping + single parity distributed across all disks
- Can survive 1 disk failure
- Capacity: (N-1) × disk size (1 disk equivalent lost to parity)
- Storage efficient: Only 1 disk equivalent lost to redundancy
- Write penalty: Each write requires 4 I/O operations (read old data, read old parity, write new data, write new parity)

RAID Level 6 : Striping with Double Distributed Parity

- Striping + dual parity (different algorithms: P+Q parity)
- Can survive 2 simultaneous disk failures
- Capacity: (N-2) × disk size (2 disk equivalent lost to parity)
- Higher write penalty: Even more calculations than RAID 5
- Lower storage efficiency: Two disks lost to parity

RAID Level 10 (1+0) : Mirroring + Striping

- First mirrors disks (RAID 1), then stripes across mirrors (RAID 0)
- Minimum 4 disks (2 pairs)
- Capacity: 50% of total disk space (like RAID 1)
- Minimum 4 disks required
- Cost per GB is highest

Hashing

Hashing is a database indexing technique that maps search keys to fixed-size values (hash values) using a hash function. It provides O(1) average-case access time for exact match queries, making it extremely efficient for equality-based operations.

Types of Hashing

1. Static Hashing
    a. Predetermined buckets at start
    b. Collision handled by overflow chains
    c. Use when dataset size is known, fixed, and small
2. Dynamic Hashing
    a. Buckets split when full, merge when empty
    b. Maintains consistent performance regardless of data size
    c. Use when dataset grows unpredictably, needs consistent performance

Static Hashing

- A fixed number of buckets (storage units, typically a disk block) are allocated.
- A hash function h(K) is applied to the search key K of a record.
- The result determines which bucket the record belongs to.

Formula: Bucket_Address = h(K)

Inserting a Key

Steps:

- Apply hash function
- Go to target bucket
- Insert record if space available

Example:

```
h(key) = key mod 10
```

Searching a Key

Steps:

1. Apply hash function → get bucket number
2. Go directly to that bucket
3. Search inside the bucket

Example:

If h(23) = 23 mod 10 = 3

→ Go to bucket 3

→ Search inside bucket 3

What is a Collision?

When:

- Two keys hash to the same bucket
- OR bucket becomes full

What is an Overflow Bucket?

An extra bucket linked to the original bucket when it becomes full.

Problems with Overflow Buckets

- Slower search (must traverse chain)
- Degrades performance
- Too many overflow buckets reduce efficiency

This is the main weakness of static hashing.

Dynamic Hashing

It allows the hash function to change dynamically as the database grows or shrinks. It adds or removes buckets on demand.

Key Idea:

- The hash function generates a large number of bits (e.g., a 32-bit integer).
- We only use a certain number of bits (a pseudokey) to locate the bucket.
- As the file grows, we use more bits to distinguish between records, splitting buckets when they overflow.

Dynamic Hashing Example

Here is a simple, step-by-step example of how keys are distributed into buckets using their binary digits as the database grows.

Imagine we are inserting keys with the following hash values (we'll use small binary numbers for simplicity):

Key A: 1010 (Binary for 10)

Key B: 1111 (Binary for 15)

Key C: 1011 (Binary for 11)

Key D: 1100 (Binary for 12)

Key E: 1001 (Binary for 9)

Dynamic Hashing Example

STEP 1: Initial State

Global Depth (GD) = 1

Directory size = 2^1 = 2 entries

Directory (using last 1 bit):

- 0 → Bucket 0
- 1 → Bucket 1

Insert keys:

- Key A: 1010 → last bit = 0 → Bucket 0
- Key B: 1111 → last bit = 1 → Bucket 1

Buckets:

- Bucket 0: A (1010)
- Bucket 1: B (1111)

STEP 2: Insert Key C (1011)

Key C: 1011 → last bit = 1 → Bucket 1

Bucket 1 now contains:

- B (1111)
- C (1011)

Bucket 1 is full (capacity = 2), but no overflow yet.

STEP 3: Insert Key D (1100)

Key D: 1100 → last bit = 0 → Bucket 0

Bucket 0 now contains:
A (1010)
D (1100)

Bucket 0 is full.

Dynamic Hashing Example

STEP 4: Insert Key E (1001)

Key E: 1001 → last bit = 1 → Bucket 1

Bucket 1 currently:
B (1111)
C (1011)

Trying to insert E causes *OVERFLOW* in Bucket 1.

Currently:

Global Depth = 1

Local Depth of Bucket 1 = 1

Since Local Depth = Global Depth, we must:

Increase Global Depth to 2

Double the directory size

STEP 5: Directory Doubling

Global Depth = 2

Directory size = 4 entries

Directory now uses last 2 bits:

00 → Bucket 0

01 → Bucket 1

10 → Bucket 0

11 → Bucket 1

Now split Bucket 1 into two buckets.

Increase local depth of split buckets to 2.

Dynamic Hashing Example

STEP 5: Directory Doubling

Global Depth = 2
Directory size = 4 entries

Directory now uses last 2 bits:

00 → Bucket 0
01 → Bucket 1
10 → Bucket 0
11 → Bucket 1

Now split Bucket 1 into two buckets.

Increase local depth of split buckets to 2.

Final Structure

Global Depth = 2

Directory:

00 → Bucket 0 (A, D)
10 → Bucket 0 (A, D)
01 → Bucket 01 (E)
11 → Bucket 11 (B, C)

Bucket 0 local depth remains 1
Bucket 01 local depth = 2
Bucket 11 local depth = 2